Format contracts

This document is the single source of truth for the data exchanged between the three modules of the project. Do not change any field name or meaning without agreeing on it with the maintainer first, and bump `contract_version` when you do.

The machine-readable definitions live next to this file:

- `docs/schemas/capture.schema.json` — normalized capture
- `docs/schemas/result.schema.json` — rule application result

Both are JSON Schema draft 2020-12.

1. Normalized capture

Produced by the Capture Engine. Consumed by the Protocol engine and the UI.

```
{
  "contract_version": 1,
  "capture_id": "sha256:<hash>",
  "source_file": "captures/cap1.pcapng",
  "sessions": [
    {
      "session_id": "s1",
      "endpoints": [
        { "ip": "...", "port": 0 },
        { "ip": "...", "port": 0 }
      ],
      "directions": {
        "A_to_B": {
          "bytes_b64": "...",
          "provenance": [
            { "offset": 0, "length": 4, "packet_index": 42, "seq": 1000, "ts": "..." }
          ],
          "diagnostics": [
            { "type": "gap", "offset": 100, "length": 8 }
          ]
        }
      }
    }
  ]
}
```

Notes:

- `capture_id` is `sha256:` followed by the lowercase hex digest of the source capture file.
- `source_file` is relative to the project root.
- `bytes_b64` is the reassembled directional stream, base64 encoded. Gaps are not filled with zero bytes; the missing ranges appear in `diagnostics` with `type` = `gap`.

- `diagnostics[].type` is an open set. Stream defects use `gap` and `ambiguity`; parser notes use `non_ip`, `non_tcp`, `ip_fragment_reassembled`, `ip_fragment_incomplete`, `ip_fragment_overlap`, `truncated_packet`, `truncated_frame`, and `unsupported_linktype`. Consumers must tolerate types they do not know. Every diagnostic that points at stream bytes carries `offset` and `length`; parser notes may carry `packet_index` instead. `docs/CAPTURE_API.md` lists each type with its meaning.
- Every entry of `provenance` maps a byte range of the stream (`offset`, `length`) to the packet it came from (`packet_index`, `seq`, `ts`). `packet_index` is the index of the packet in the capture file as read.
- `ts` is the packet timestamp in epoch seconds.

2. Rule application result

Produced by the Protocol engine. Consumed by the UI.

```
{
  "contract_version": 1,
  "rule_id": "r1",
  "rule_version": 3,
  "capture_id": "...",
  "messages": [
    { "offset": 0, "length": 16, "fields": { "cmd": 4, "value": 21 }, "status": "matched" }
  ],
  "summary": { "matched": 205, "mismatched": 12, "unknown": 23, "uncovered": 0 }
}
```

Notes:

- `rule_id` and `rule_version` identify the rule that produced the result.
- `messages[].offset` and `messages[].length` locate the message in the directional stream.
- `messages[].fields` holds the decoded field values keyed by field name.
- `messages[].status` is one of the statuses defined in `result.schema.json`.
- `summary` counts messages per status.

3. On-disk project layout

Owned by the UI / Project module.

```
project.madrigal/
  manifest.json
  captures/
  logs/
  rules/
  results/
  annotations.sqlite
  reports/
```

All paths inside the manifest are relative to the project directory so that the project can be moved to another directory or machine and still open.